

C++ Interview Evaluation Report

Topic: Constructors, Destructors, and Operator Overloading

Level: Fresher (MNC Interview Simulation)

Question 1:

What is a constructor in C++?

Answer:

A constructor is a special member function that is automatically called when an object is created. It has the same name as the class and does not have any return type. Its main role is to initialize the object's data members and set up the initial state of the object.

Score: 10/10

Question 2:

What is the purpose of a constructor?

Answer:

The purpose of a constructor is to initialize the data members of an object when it is created. It allows allocation of memory.

Score: 9/10

Question 3:

How is a constructor different from a normal member function?

Answer:

A constructor has the same name as the class and does not have a return type, whereas a normal function can have any name and must have a return type. A constructor is called automatically when an object is created, while a normal member function has to be called explicitly. Constructors are mainly used for initialization, whereas normal functions perform operations on the object.

Score: 10/10

Question 4:

What are the types of constructors in C++?

Answer:

The main types of constructors in C++ are default constructors, parameterized constructors,

and copy constructors. In modern C++, there are also move constructors, but the basic three are most commonly discussed in interviews.

Score: 9/10

 Question 5:

What is a default constructor?

Answer:

A default constructor is a constructor that does not take any parameters. It initializes the object with default values. If no constructor is defined by the programmer, the compiler automatically provides a default constructor.

Score: 8.5/10

 Question 6:

What is a parameterized constructor?

Answer:

A parameterized constructor is a constructor that takes arguments. It allows the object to be initialized with specific values at the time of creation, making the initialization more flexible and meaningful.

Score: 9/10

 Question 7:

What is a copy constructor?

Answer:

A copy constructor is a constructor that initializes a new object using an existing object of the same class. It takes a reference to another object and copies its values. It is especially important when working with dynamic memory to avoid shallow copying issues.

Score: 10/10

 Question 8:

When is a copy constructor called?

Answer:

A copy constructor is called when an object is initialized using another object, when an object is passed to a function by value, or when an object is returned from a function by value.

Score: 10/10

 Question 9:

What is constructor overloading?

Answer:

Constructor overloading is the concept of having multiple constructors in the same class with different parameter lists. This allows objects to be initialized in different ways depending on the arguments provided.

Score: 10/10

 Question 10:

Can a constructor be private? Why?

Answer:

Yes, a constructor can be private. This is used to restrict object creation from outside the class. It is commonly used in design patterns like the Singleton pattern, where only one instance of the class is allowed.

Score: 10/10

 Question 11:

What is a destructor?

Answer:

A destructor is a special member function that is automatically called when an object is destroyed. Its main purpose is to clean up resources used by the object before it is removed from memory.

Score: 9/10

 Question 12:

What is the syntax of a destructor?

Answer:

A destructor has the same name as the class but is preceded by a tilde (~) symbol and does not take any parameters or return any value.

Score: 9/10

 Question 13:

When is a destructor called?

Answer:

A destructor is called when an object goes out of scope, when the delete operator is used for dynamically allocated objects, or at the end of the program for static and global objects.

Score: 10/10

 Question 14:

Can a class have multiple destructors?

Answer:

No, a class can have only one destructor because it does not take parameters and therefore cannot be overloaded.

Score: 9/10

 Question 15:

What is the main purpose of a destructor?

Answer:

The main purpose of a destructor is to release resources such as dynamically allocated memory, close files, and clean up any external connections to prevent memory leaks.

Score: 10/10

 Question 16:

What happens if we don't define a destructor?

Answer:

If a destructor is not defined, the compiler automatically provides a default destructor. However, it only performs basic cleanup and does not free dynamically allocated memory, which can lead to memory leaks if not handled properly.

Score: 10/10

 Question 17:

Difference between constructor and destructor?

Answer:

Constructors help by allocating and initializing resources when an object is created, while

destructors ensure that all allocated resources are properly released when the object is destroyed. This prevents memory leaks and ensures efficient resource management.

Score: 7.5/10

 Question 18:

What happens when an object is created?

Answer:

When an object is created, memory is allocated for it, and the constructor is automatically called to initialize its data members.

Score: 8/10

 Question 19:

What happens when an object goes out of scope?

Answer:

When an object goes out of scope, its destructor is automatically called, and the memory allocated to the object is released.

Score: 8.5/10

 Question 20:

How do constructors and destructors help in memory management?

Answer:

Constructors help by allocating and initializing resources when an object is created, while destructors ensure that all allocated resources are properly released when the object is destroyed. This prevents memory leaks and ensures efficient resource management.

Score: 10/10

 Question 21:

What is dynamic memory allocation and how is it related?

Answer:

Dynamic memory allocation is the process of allocating memory at runtime using operators like new and deallocating it using delete. Constructors are often used to allocate such memory, and destructors are used to free it, ensuring proper memory management.

Score: 10/10

 Question 22:

What is operator overloading?

Answer:

Operator overloading is a feature that allows existing operators to be given new meanings when applied to user-defined data types like objects.

Score: 8.5/10

 Question 23:

Why do we use operator overloading?

Answer:

Operator overloading is used to make code more readable and intuitive by allowing operators to be used with objects just like built-in data types. It improves clarity and simplifies expressions.

Score: 10/10

 Question 24:

Which operators cannot be overloaded?

Answer:

Operators such as scope resolution (::), member access (.), pointer-to-member (*), conditional (?), and sizeof cannot be overloaded.

Score: 10/10

 Question 25:

Syntax for operator overloading?

Answer:

```
return_type operator operator_symbol(argument)
```

Score: 7.5/10

 Question 26:

Unary vs Binary operators?

Answer:

Unary operators work with a single operand, such as increment or decrement operators, while binary operators work with two operands, such as addition or subtraction.

Score: 8.5/10

 Question 27:

Overloading + operator?

Answer:

The + operator is overloaded by defining a function named operator+ in the class. This function takes another object as a parameter and returns a new object whose value is the result of adding the two objects' data members.

Score: 10/10

 Question 28:

Assignment operator overloading?

Answer:

Yes, the assignment operator can be overloaded. It is often overloaded to perform deep copying of objects, especially when the class uses dynamic memory, to avoid issues like shallow copying.

Score: 10/10

 Question 29:

Purpose of this pointer?

Answer:

The this pointer refers to the current object that is invoking the operator. It is used to access the data members of the current object and helps differentiate between the current object and the parameter object.

Score: 10/10

 Question 30:

Advantages & limitations of operator overloading?

Answer:

Operator overloading makes code more readable and natural by allowing us to use operators like + or == with objects instead of function calls. However, if not used carefully, it

can make the code confusing, and we cannot create new operators or overload certain ones like `.` or `::`.

Score: 10/10

Final Evaluation

Score Summary:

- Most answers: 9–10/10
 - Few answers: 7.5–8.5/10
-

 Final Score: 9.3 / 10 = 93/100

PROOF:

 **Question 26:**

What is the difference between unary and binary operator overloading?

 Your answer.

    

Unary operators work with a single operand, such as increment or decrement operators, while binary operators work with two operands, such as addition or subtraction.

Good — that's correct and clear, but slightly basic.

 Evaluation: 

Good — that's correct and clear, but slightly basic.

 **Evaluation:**

-  Correct definition of unary vs binary
-  Gave examples (increment, addition)
-  Easy to understand
-  Could connect it specifically to operator overloading context

 **Score for this question: 8.5/10** 

🔪 Question 25:

What is the syntax for operator overloading?

👉 Your answer:

📄 🏠 🗨️ ⬆️ ↺️ ...

```
return_type operator operator_symbol(argument)
```

Good — you've got the **core idea**, but it's a bit incomplete for an interview.

✅ Evaluation:

- ✓ Correct basic syntax idea



```
return_type operator operator_symbol(argument)
```

Good — you've got the **core idea**, but it's a bit incomplete for an interview.

✅ Evaluation:

- ✓ Correct basic syntax idea
- ✓ Included `operator` keyword and symbol
- | Missing class context and function structure

★ Score for this question: 7.5/10

